Bringing SQL Row Pattern Matching MATCH_RECOGNIZE Operator to Pandas DataFrames

Monier Lawande[a], Mohamed Elhelw[a], Ahmed Awad[b,c]

[a] *Center for Informatics Science (CIS), School of Information Technology and Computer Science (ITCS), Nile University, Giza, Egypt*
[b] *British University in Dubai, Dubai, UAE*
[c] *Cairo University, Giza, Egypt*

Abstract

The SQL standard introduced the MATCH_RECOGNIZE operator in 2016 to support row pattern recognition, enabling the detection of ordered sequences of rows in a table that match a user-defined pattern. This capability is useful in diverse domains—for example, a financial institution may detect a sequence of small deposits followed by rapid transfers and large withdrawals, a known indicator of potential money laundering, or climate analysts may identify heatwaves as three or more consecutive days with maximum temperatures exceeding 40°C. MATCH_RECOGNIZE is now supported by several leading database systems, such as Oracle, Snowflake, and Flink. In this paper, we present a prototype that brings this functionality to Pandas DataFrames, a widely used in-memory data structure in the Python data science ecosystem. Our prototype parses SQL-based pattern queries that execute MATCH_RECOGNIZE() queries directly from an SQL string on a pandas DataFrame. The system supports SQL:2016 row pattern recognition (RPR) semantics, including PARTITION BY, ORDER BY, PATTERN, DEFINE, and MEASURES for offline analytics in Python. constructs an abstract syntax tree (AST), translates patterns into non-deterministic finite automata (NFA), and executes them efficiently over DataFrames. We evaluate the engine against Trino 473 and Oracle 21c EE on the Amazon UK Products dataset, running five pattern families over six sizes up to 2,222,742 rows. The three systems return identical results in all 30 pattern–size cells, and the engine averages 0.17 seconds per cell against 1.04 for Oracle and 1.98 for Trino. A scalability test on the Amazon Reviews 2023 corpus extends the input to 227,899,533 rows, which the engine completes on every pattern family. The declarative form also reduces code size compared to native Pandas implementations, extending the convenience of SQL pattern matching to Python users while preserving correctness equivalent to that of established SQL engines.

Keywords: Pattern Matching, MATCH_RECOGNIZE, Pandas, SQL, Finite Automata, Data Engineering

1. Introduction

Pattern matching is the process of analyzing a sequence of data points, which can be rows in a table or events in a stream. Pattern matching plays a crucial role in data analysis, especially for identifying patterns and shapes or specific sequences of events or behaviors in areas like identifying chains of transactions related to money laundering in finance[1], revealing fraudulent behaviours of users in security logs[2]. Patterns in data can be described using regular-expression-like syntax, like those commonly used in textual analysis. However, unlike traditional text-based regular expressions, these patterns are applied at the row level, enabling analysts to match and extract meaningful sequences across entire records rather than individual characters.

The MATCH_RECOGNIZE (MR) clause was introduced in the SQL:2016 standard [3, 4], enabling users to define and search for patterns within ordered sequences of rows directly in SQL queries. The MATCH_RECOGNIZE SQL clause enables advanced pattern-based querying over data tables. Although it was standardized in SQL:2016 [3, 4], a few DBMS support it. For example, among well-known traditional RDBMSs, only Oracle[5, 6] supports the MR operator. In the big data analytics systems, Trino [7, 8], Snowflake [9], and Apache Flink [10] support the MR operator. Cloud warehouses have since followed: BigQuery documents the clause [11], and Databricks introduced beta support in Runtime 19.0 [12]. By contrast, PostgreSQL 18 still lists the related SQL features as unsupported [13], while DuckDB lists native MATCH_RECOGNIZE as planned work [14].

Without the support for the declarative MR clause, tasks requiring detecting multi-row patterns often require complicated combinations of SQL window functions, self-joins, or even procedural extensions and user-defined functions to loop through rows and maybe add some external programming outside the database [15]. As soon as the pattern grew beyond two or three steps, the SQL statement expanded into multiple lines full of row-number calculations, timestamp comparisons, and complex JOIN conditions. Not only is the code hard to write and even harder to maintain, but each extra join or analytic window often means an extra table scan, leading to poor performance. In practice, many teams either accepted these limitations or abandoned SQL for external scripts and ETL pipelines just to get reliable pattern matching SQL expanded into multiple lines [16].

Email addresses: m.ashraf2288@nu.edu.eg (Monier Lawande), melhelw@nu.edu.eg (Mohamed Elhelw), ahmed.awad@buid.ac.ae (Ahmed Awad)

With the introduction of MATCH_RECOGNIZE in the SQL:2016 standard, rather than a multitude of lines of code and using window functions and self-joins, Instead, a single declarative MATCH_RECOGNIZE statement that clearly defines the data partitions, ordering, pattern, and output measures. Simplifying both the syntax and readability of pattern-matching queries, MATCH_RECOGNIZE clause enables users to organize their data into logical groups using `PARTITION BY`, specify the sequence of row analysis using `ORDER BY`, and define the actual pattern using the `PATTERN` section, which uses regular expression-like syntax. Logical conditions that define each pattern element are clearly stated in the `DEFINE` clause, while the `MEASURES` clause determines what information is extracted from each detected match. This structure allows complex sequence patterns to be described clearly and concisely. It ensures that detecting, classifying, and summarizing sequences can be performed efficiently and consistently within standard SQL queries, such as trends, repeated behaviors, or sudden changes, directly within standard SQL queries [17].

Before introducing the MATCH_RECOGNIZE method in Pandas, detecting multi-row patterns such as consecutive upward trends, cyclical oscillations, or conditional event sequences required manual integration of multiple low-level operations. A typical workflow involved creating multiple shifted versions of each column (df['value'].shift(-1), shift(-2), etc.), building complex boolean masks, filtering or using groupby().apply() to stitch subsequences back together, and then pivoting or renaming columns to get the desired output. This approach was verbose (often 10+ lines for modest patterns), prone to off-by-one or alignment bugs, and hard to extend to longer or conditional patterns.

For example, an e-commerce platform that tracks all customer purchases: the marketing team wants to find customers who are decreasing their spending money by buying progressively cheaper items in three consecutive purchases, by detecting three consecutive purchases using pandas functions for pattern (A B C).

```
import pandas as pd
df['order_date'] =
    pd.to_datetime(df['order_date'])
# Sort for partition
df = df.sort_values(['customer_id',
    'order_date']).reset_index(drop=True)
def match_pattern_ABC(group: pd.DataFrame):
    """Find matches where B.price < A.price and
        C.price < B.price."""
    matches = []
    n = len(group)
    for i in range(n - 2):  # need at least 3
        rows for A,B,C
        A = group.iloc[i]
        B = group.iloc[i + 1]
        C = group.iloc[i + 2]

        if (B['price'] < A['price']) and
            (C['price'] < B['price']):
            matches.append({
                'customer_id': A['customer_id'],
                'start_date': A['order_date'],
                'end_date': C['order_date'],
                'bottom_price': C['price']
            })
    return matches
# Apply per partition
all_matches = []
for cust_id, grp in df.groupby('customer_id'):
    all_matches.extend(match_pattern_ABC(grp))

result_df = pd.DataFrame(all_matches)
print(result_df)
```

Listing 1: Apply `MATCH_RECOGNIZE` using pandas functions

After integrating MATCH_RECOGNIZE into Pandas, the complex operations that were applied before were eliminated. In a single statement, it is possible to define partitioning column(s), ordering key, pattern grammar (e.g., "A B C"), logical definitions for each step, and the summary or detail measures that will be returned. The method returns a clean, annotated DataFrame of full matches. Additionally, it is simple to expand to more intricate patterns, varying lengths, or conditional patterns; simply update the pattern string or definitions to align with specifications. Using the same declarative SQL syntax can be effectively utilized for multiple analytical use cases [18].

The MATCH_RECOGNIZE clause:

```
SELECT <columns to return>
FROM <table_name>
MATCH_RECOGNIZE
(
  [ PARTITION BY <columns to group data by>
    ]
  [ ORDER BY <columns to sort each group> ]
  [ MEASURES <expressions to define output
    values> ]
  [ ONE ROW PER MATCH | ALL ROWS PER MATCH ]
  [ AFTER MATCH <skip to option> ]
  [ PATTERN ( pattern_expression ) ]
  [ SUBSET <combine multiple variables into
    one> ]
  DEFINE <rules for each pattern variable>
);
```

Core Components of Row Pattern Recognition:

Row pattern recognition in SQL, as standardized in SQL:2016, detects patterns in sequential data through the `MATCH_RECOGNIZE` clause. The foundational architecture establishes independent pattern matching scopes via `PARTITION BY` clauses and defines temporal ordering through `ORDER BY` specifications within each partition. Pattern definitions utilize regular expression-inspired syntax (e.g., A B+ C?) to express complex event sequences, incorporating fundamental operations such as concatenation, alternation, and quantified repetition through standard quantifiers (`*`, `+`, `?`, `{n,m}`). The `DEFINE` clause establishes boolean predicates for pattern variables, enabling the incorporation of navigation functions (`FIRST`, `LAST`, `PREV`, `NEXT`) that facilitate temporal and positional analysis of data relationships with appropriate boundary condition handling. The `MEASURES` clause enables the extraction of computed values from successful pattern matches, supporting standard functions including `CLASSIFIER()` for pattern variable identification and `MATCH_NUMBER()` for match sequence enumeration, with dual semantic evaluation modes:

RUNNING for incremental computation and FINAL for complete match evaluation. SUBSET variables provide pattern abstraction capabilities through hierarchical variable grouping mechanisms. Pattern anchors (^, $), exclusion blocks ({- ... -}), and PERMUTE extend the pattern language further, and a variable used in PATTERN without a DEFINE predicate is treated as implicitly true. Match control policies, implemented via AFTER MATCH SKIP directives, govern result cardinality and pattern overlap behavior while determining continuation strategies for subsequent pattern detection iterations, ensuring deterministic matching behavior in compliance with SQL:2016 specifications.

For example, an e-commerce platform that tracks all customer purchases, the marketing team wants to find customers who are decreasing their spending money by buying progressively cheaper items in three consecutive purchases, by detecting three consecutive purchases using MATCH_RECOGNIZE for the pattern (A B C).

```
import pandas as pd
from src.executor.match_recognize import
    match_recognize
df['order_date'] =
    pd.to_datetime(df['order_date'])
# Define the MATCH_RECOGNIZE query
query = """
SELECT customer_id, start_date, end_date,
    bottom_price
FROM orders
MATCH_RECOGNIZE (
  PARTITION BY customer_id
  ORDER BY order_date
  MEASURES
    A.order_date AS start_date,
    C.order_date AS end_date,
    C.price      AS bottom_price
  PATTERN (A B C)
  DEFINE
    B AS B.price < A.price,
    C AS C.price < B.price
);
"""
# Execute and print results
output_df = match_recognize(query, df)
print(output_df)
```

Listing 2: Using MATCH_RECOGNIZE

2. Background and Related Work

The SQL:2016 MATCH_RECOGNIZE clause is supported by many leading SQL engines natively, like Oracle[5, 6], Snowflake[9], and Trino [7, 8] and others while still under consideration to implement this feature, like PostgreSQL [19], MariaDB [20], and Microsoft SQL Server [21], To use MATCH_RECOGNIZE in these systems without native support, users must emulate row-pattern matching using a combination of window functions, user-defined aggregates, or external procedural layers—approaches that are often more complex, less efficient, and harder to maintain, or external processing layers [22, 23]. A different route is translation: the database group at the University of Tübingen rewrites row-pattern queries into recursive common table expressions that DuckDB already executes [15], which avoids adding a native operator but still requires a second engine. The landscape continues to move: BigQuery documents the clause [11] and Databricks offers it in beta [12], while PostgreSQL 18 [13] and DuckDB [14] still list it as absent or planned.

2.1. Pandas

Pandas is a popular, open-source Python library for data analysis and manipulation. Its straightforward installation process and strong community support make it a good choice for students, researchers, and professionals who frequently need to analyze data without using complicated systems like Oracle, Trino, or Snowflake. For everyday tasks, Pandas offers straightforward methods for managing data sequences, filtering, grouping, and performing basic time-series analyses. However, it has difficulty with advanced pattern detection in contrast to SQL's MATCH_RECOGNIZE clause. Pandas lacks a straightforward way to express complex patterns, users must create their own solutions using functions like rolling, groupby, or shift. These custom approaches can become complicated and error-prone, especially with advanced patterns. Pandas is highly popular due to ease of setup and use, absence of licensing fees, and smooth integration within the larger Python libraries. This makes it great for quickly developing and testing ideas for typical pattern detection needs.

2.2. Existing Implementations

Oracle [5, 6], Trino [7, 8], Snowflake [9], and Flink [10] all provide advanced solutions for SQL-based pattern matching, but they each come with important practical challenges that users need to keep in mind. Oracle provides a comprehensive and robust implementation of the MATCH_RECOGNIZE clause, well-suited for complex, enterprise-scale analytics. However, this power comes with high operational costs (including licensing fees). Trino, an open-source alternative, supports MATCH_RECOGNIZE with a syntax close to the SQL:2016 standard and performs well in distributed data environments. It is more accessible than Oracle in terms of deployment and cost, but still requires considerable infrastructure management and may lack some of Oracle's advanced optimizations. Snowflake is a cloud-native platform that is scalable and provides a user-friendly MATCH_RECOGNIZE feature for identifying common event patterns. However, offers limited support for advanced clauses and has computational costs that relate to the size of the warehouse. Apache Flink is designed for real-time stream processing, allowing rapid detection of event patterns in continuous data flows. While Flink is highly scalable and effective for high-velocity data, it introduces its own learning curve and is less familiar to users accustomed to traditional, batch-based SQL systems [24, 25].

A major limitation shared by all these platforms is the complexity of integrating SQL-based pattern matching with machine learning workflows and other analytics tools [26, 27]. Typically, data must be exported from the database to external environments for ML modeling, resulting in data duplication, consistency challenges, and more complicated data pipelines [28]. Keeping track of multiple versions of the same dataset across systems also increases the risk of errors and inefficiency. For many users especially those with modest analytics needs or who work independently, installing and managing large-scale systems like Oracle, Trino, Snowflake, or Flink can be overly complicated for some users. These users may only need to perform occasional pattern detection and would prefer to avoid the overhead of learning different functions or dealing with varying syntax for similar queries across platforms [29]. Additionally, for datasets that fit comfortably in memory, tools like Pandas offer a faster and more direct alternative, eliminating the latency of disk-based operations common to larger database engines[30, 31]. Although Oracle, Trino, Snowflake, and Flink each have unique strengths for SQL pattern matching in large-scale or real-time scenarios, they introduce extra costs, infrastructure requirements, and integration issues that can be prohibitive for everyday users. These limitations highlight the value of Pandas, as it offers simpler, memory-based tools and unified workflows, where pattern matching and data analytics can be performed seamlessly and without unnecessary duplication or complexity.

3. System Design and Methodology

Our `MATCH_RECOGNIZE` implementation follows a modular, five-stage architecture specifically designed for SQL row pattern recognition. The engine executes SQL queries directly on `pandas` DataFrames [32], while fully preserving SQL:2016 semantics [3, 4, 33]. The pipeline consists of five phases: parsing, semantic validation, logical plan construction, pattern compilation, and execution (Fig. 1).

In the parsing stage, the SQL query is transformed into an abstract syntax tree (AST) that captures both the `SELECT`/`FROM` clauses and `MATCH_RECOGNIZE` subclauses [34, 35]. Semantic validation then checks schema references, variable usage, predicates, skip policies, and output modes. Compliance rules are applied to ensure standard conformance, correct use of navigation functions, consistent measure semantics, and robust handling of edge cases such as empty matches, partition boundaries, and skip behaviors.

The validated plan is translated into a logical plan composed of modular operators for partitioning, sorting, matching, measuring, and skipping. From this plan, the system compiles a nondeterministic finite automaton (NFA) with guarded transitions that support quantifiers, alternation, and `PERMUTE` constructs [36]. The NFA is constructed using Thompson's construction, extended for SQL:2016 semantics by embedding predicates from the `DEFINE` clause, minimizing epsilon transitions, and applying subset construction with state deduplication to mitigate exponential growth. Because subset construction is exponential in the worst case, the compiler is guarded:

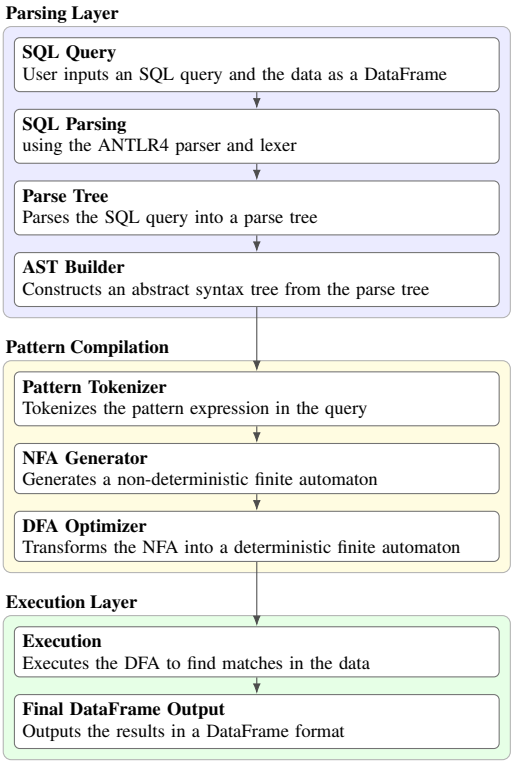


Figure 1: System architecture.

`PERMUTE` is rejected above eight variables or an estimated 50,000 branches, and DFA construction stops at a ceiling of 50,000 states. Exceeding any guard raises an explicit compilation error, so the engine never executes or caches a partial automaton.

Finally, execution is performed over ordered partitions using the compiled automaton. Matches are materialized with the correct `RUNNING`/`FINAL` semantics. The evaluation engine manages context separation, applies navigation functions (`FIRST`, `LAST`, `PREV`, `NEXT`) with partition awareness, and computes aggregates such as `COUNT`, `SUM`, and `AVG` across pattern variables. This ensures full compliance with SQL:2016 while producing results directly as `pandas` DataFrames.

3.1. SQL Parsing and AST Construction

The process starts by analyzing the user's SQL query, identifying its logical components. This proceeds through a parsing process that explores the query, recognizes its main components such as grouping (`PARTITION BY`), ordering (`ORDER BY`), the pattern to be searched, and any specific conditions or output metrics, and then organizes these into an Abstract Syntax Tree (AST). The AST provides a clear and structured representation of the query. This approach eases query validation and prepares it for pattern matching in the following stages.

3.2. Pattern Translation to Finite Automata

Once the AST is completed, the system focuses on the pattern component described in the SQL query. This pattern is then converted into a finite automaton, which defines a set of

states and transitions for detecting complex sequences or behaviors in the data. This process begins by constructing a nondeterministic finite automaton (NFA) from the pattern and then, for improved efficiency, transforms it into a deterministic finite automaton (DFA) [25]. The DFA enhances the matching process by removing ambiguity in transitions, enabling the system to scan data sequences quickly and accurately. This translation process enables the system to handle a wide variety of pattern features, such as repetitions, alternatives, and specific sequences, in a unified and efficient way. The resulting automaton acts as the core mechanism for scanning and detecting patterns within the data.

3.3. Execution Engine on Pandas

The execution phase uses the constructed DFA to perform pattern matching directly over input data represented as Pandas DataFrames. The engine begins by partitioning and ordering the data according to the query's specifications. It then applies the DFA to scan the dataset, identifying all sequences that fit the defined pattern. For every detected match, the engine computes any requested measures or calculations and compiles the results into a DataFrame formatted to mirror standard SQL query outputs. By utilizing Pandas, the system retains both flexibility and accessibility, smoothly incorporating advanced pattern matching into conventional Python data analysis workflows.

4. Implementation and Optimization

4.1. ANTLR4 Grammar and AST Construction

Our system uses a customized SQL grammar, extended from Trino's SQL that fully supports the SQL:2016 `MATCH_RECOGNIZE` clause. This grammar was defined by ANTLR4, including complex patterns, nested patterns, alternation, quantifiers, grouping, and exclusions [37]. The parser's output is a detailed parse tree that captures the syntactic structure of each SQL query[35],[34].

4.2. Finite Automata

Pattern Tokenization, Pattern expressions are tokenized using a recursive descent tokenization algorithm that supports anchors, grouping, quantifiers, and nested constructs.

NFA Construction, We use Thompson's construction algorithm to translate pattern tokens into a Non-deterministic Finite Automaton (NFA). Each state captures transitions, variable bindings, anchors, and metadata for advanced features. Quantifiers (`*`, `+`, `?`, `{n,m}`) are directly mapped to sub-automata [25].

DFA Construction and Optimization, The NFA is systematically converted into a Deterministic Finite Automaton (DFA) using subset construction, drastically reducing the number of active states during execution. Using optimization steps like state minimization, transition indexing, and merging equivalent states results in a compact and performant automaton. The resulting DFA supports efficient row-by-row evaluation while preserving pattern semantics and priorities and custom conditions for accurate pattern semantics [38, 39, 25].

4.3. Execution

The matching engine applies the DFA sequentially over each partition of the DataFrame with specialized handling for anchors, skip modes, and edge cases. The executor coordinates the entire pattern matching process, integrating parsing, automata construction, match identification, and result formatting.

Condition and Measure Evaluation, A dedicated evaluation subsystem translates SQL pattern conditions and measures into executable Python code that supports navigation functions and aggregations with SQL:2016 compliant NULL semantics for both running and final calculations.

5. Experimental Evaluation

5.1. Evaluation Methodology

5.1.1. Evaluation Criteria and Metrics

We evaluate the proposed `MATCH_RECOGNIZE` engine through five criteria. First, `output correctness`: results are compared with Trino 473 and Oracle 21c EE by row-level equality across multiple pattern types, ensuring identical matching, pattern alignment, and measure calculations. Second, `semantic coverage`: following SQL:2016 `MATCH_RECOGNIZE` semantics—including `PERMUTE` patterns, exclusions, quantifiers, navigation functions, and output modes—and compatibility with standard SQL semantics [3, 4, 32]. Third, `developer effort (code size)`: the lines required to express equivalent logic in idiomatic `pandas`; baselines typically need substantially more code (e.g., chained `groupby`/`shift`/joins) than a single `MATCH_RECOGNIZE` statement, yielding reduced LOC in line with the row pattern recognition (RPR) goal of lowering implementation effort [3, 4, 33, 40]. Fourth, `performance`: client-observed execution time, throughput, and memory for every pattern–size cell, where each cell is the mean of twenty measured runs after five warm-up runs and outliers are removed by the 1.5× interquartile range rule. Fifth, `scalability`: how execution time and memory grow as the input grows, measured on real data up to 227,899,533 rows.

5.1.2. Environment and Configuration

The experiments were performed on a workstation running Ubuntu 24.04.4 LTS with an Intel Core i711800H (16 cores at 2.30 GHz) and 64 GB of DDR4 RAM at 3200 MHz. The software setup uses Trino 473 (baseline SQL engine); Java 11.0.25 LTS and Python 3.12.7; Python libraries including `pandas` 2.3.3, `numpy` 2.1.3, `antlr4-python3-runtime` 4.13.2, `antlr4-tools` 0.2.1, `pytest` 8.3.4, Jupyter, `unittest`, `matplotlib` 3.10.0, `plotly` 5.24.1, and `seaborn` 0.13.2; and standard libraries such as `typing`, `dataclasses`, `re`, `operator`, `math`, `itertools`, `collections`, `enum`, `time`, `sys`, `os`, `logging`, and `traceback`. `MATCH_RECOGNIZE` implementation using ANTLR4 for parsing Trino SQL, `pandas` for DataFrame management, and custom AST/NFA/DFA modules for pattern detection.

5.2. Dataset and Test Patterns

We use an Orders table to demonstrate and validate pattern-matching queries. The dataset comprises three attributes[8]:

`customer_id` (VARCHAR), which identifies the customer (two distinct values: `cust_1` and `cust_2`); `order_date` (DATE), the date the order was placed (ranging from May 11, 2020 to May 18, 2020); and `price` (INTEGER), the purchase amount on that date. Table 1 shows a sample of data from eight records:

Since each customer appears multiple times in chronological order with `cust_1` spanning five dates and `cust_2` spanning three, this table allows us to test pattern-matching queries (e.g., sequences of rising/falling prices per customer). This dataset structure enables comprehensive validation of partitioning, ordering, and pattern detection. In our experiments, we load this data via a simple SQL VALUES clause and verify that our `MATCH_RECOGNIZE` implementation produces exactly the same matched rows and measures as Trino's `MATCH_RECOGNIZE`.

```
CREATE TABLE memory.default.orders (
    customer_id VARCHAR,
    order_date DATE,
    price INTEGER
);
```

Listing 3: Creating the in-memory table in Trino

```

INSERT INTO memory.default.orders
VALUES
    ('cust_1', DATE '2020-05-11', 100),
    ('cust_1', DATE '2020-05-12', 200),
    ('cust_2', DATE '2020-05-13',   8),
    ('cust_1', DATE '2020-05-14', 100),
    ('cust_2', DATE '2020-05-15',   4),
    ('cust_1', DATE '2020-05-16',  50),
    ('cust_1', DATE '2020-05-17', 100),
    ('cust_2', DATE '2020-05-18',   6);
```

Listing 4: Inserting data into the table

Amazon UK Products.. The performance benchmark uses the Amazon UK Products dataset [41], roughly 2.2 million product records from Amazon's UK marketplace in a single CSV file. Four columns are used: `stars`, `reviews`, `price` (GBP), and `categoryName`. A derived `category` column maps the price into five bands: A (≤£10), B (£10–25), C (£25–50), D (£50–100), and E (>£100). Each row carries a sequential `seq_id` in source order.

Amazon Reviews 2023.. The scalability test needs far more rows than the product table provides, so it uses the Amazon Reviews 2023 corpus [42] from the McAuley Lab at UC San Diego, roughly 571 million genuine review events. Only the pattern-relevant fields are taken: `user_id`, `parent_asin`, `rating`, and `timestamp`; review text, images, and product metadata are excluded. Five categories are merged in global timestamp order, and each review's 1–5 rating is mapped to the same A–E alphabet, so the same pattern families apply to both datasets.

Test patterns.. All patterns run on the derived `category` column, where A is budget (≤£10), B low-mid (£10–25), C mid (£25–50), D upper-mid (£50–100), and E premium (>£100). On the Amazon Reviews corpus the same A–E alphabet is derived from the 1–5 rating. Table 2 provides detailed specifications of all five SQL patterns used in the evaluation, including their formal syntax and intended detection goals. Each pattern represents a different complexity level and use case, ranging from simple sequential matching to complex nested structures with quantifiers and alternation operators.

5.3. Complexity Analysis

Costs are stated in asymptotic notation over five symbols: n input rows, m pattern symbols, d `DEFINE` variables, k matches, and ℓ the average number of rows per match materialised in the output, so $\ell = 1$ under `ONE ROW PER MATCH`. Running a query does five kinds of work: parsing, automaton construction, ordering, condition checking, and matching with output construction.

Parsing and automaton construction depend on the pattern rather than on the data. NFA construction is $O(m)$, and the NFA-to-DFA conversion is $O(2^m)$ states in the worst case but a handful in practice; for a fixed pattern this is a one-time constant, and the compiled automaton is cached across queries. Ordering is the only data-dependent cost above linear: a comparison sort costs $O(n \log n)$ per partition, but the engine first checks in $O(n)$ whether the rows already follow the requested order and skips the sort when they do. Condition checking prepares one Boolean result per row and variable on the compiled row-local path, costing $O(nd)$, and matching with output construction then costs $O(n + k\ell)$ on the default-skip path.

Two independent axes follow. Along the data axis, with the pattern fixed, the engine is $\Theta(nd + n + k\ell) = \Theta(n)$ when the input is already ordered, as it is in the benchmark, and $O(n \log n)$ when a sort is required, because d is a property of the query rather than of the table and $k\ell = O(n)$ while matches do not overlap. Along the pattern axis, with the data fixed, three constructs are expensive in m rather than in n: the NFA-to-DFA conversion, `PERMUTE`, which expands to $O(m!)$ branches, and a `DEFINE` predicate that reads the match built so far, which cannot use the row-local Boolean matrix and may require the preference-ordered exact search. Each of the three is either paid once or held in check by the explicit compiler guards, so none of them appears as a growth curve in the data size. The measured times reported below follow the ordered case.

5.4. Empirical and Cross-System Analysis

5.4.1. Performance results using real dataset

Two campaigns are run, one on each real dataset. The first measures performance on the Amazon UK Products data over six nested prefixes: 100K, 200K, 400K, 800K, 1.6M, and the full 2,222,742 rows. The five pattern families of Table 2 run at every size, giving 30 pattern–size cells per system, and Trino

Table 1: Sample "orders" dataset (`customer_id`, `order_date`, `price`)

customer_id	order_date	price
cust_1	2020-05-11	100
cust_1	2020-05-12	200
cust_2	2020-05-13	8
cust_1	2020-05-14	100
cust_2	2020-05-15	4
cust_1	2020-05-16	50
cust_1	2020-05-17	100
cust_2	2020-05-18	6

Table 2: SQL Pattern Definitions and Detection Goals

Pattern Name	SQL Pattern	Description & Goal
simple_sequence	`A+ B+`	**Goal:** Detect transitions from budget to low-mid price listings. **Use:** Identify runs where budget products (≤£10) are immediately followed by low-mid products (£10–25), useful for detecting problematic product sequences or data quality issues in listings.
alternation	`A (B|C)+ D`	**Goal:** Detect budget-to-upper-mid transitions through mixed middle bands. **Use:** Find sequences that start with a budget product, continue with a mix of low-mid and mid products (£10–50), and end with an upper-mid product (£50–100). Useful for analyzing quality improvement patterns in product browsing sequences or sorted lists.
quantified	`A{1,5} B* C+`	**Goal:** Detect bounded budget runs followed by mid-band products. **Use:** Find sequences with 1–5 budget products, optionally followed by low-mid products, then one or more mid products (£25–50). Enforces minimum unrated product counts for identifying problematic product clusters.
optional_pattern	`A+ B? C*`	**Goal:** Flexible budget-run detection with optional band transitions. **Use:** Broad matching of budget runs with an optional low-mid product and optional mid products. High recall for various low-quality sequence scenarios in e-commerce data.
complex_nested	`(A|B)+ (C{1,3} D*)+`	**Goal:** Nested grouping of low-price runs followed by mid/upper-mid groups. **Use:** Detect runs of low-priced products (≤£25) followed by one or more groups of 1–3 mid products with optional upper-mid products. Useful for multi-level quality grouping and advanced sorting pattern detection in product listings.

473 and Oracle 21c EE execute the same query bodies as references. Each cell is the mean of twenty measured runs after five warm-up runs, with outliers removed by the 1.5× interquartile range rule. All 90 cells completed, and the three systems returned the same normalised result set in every cell.

The second campaign tests scalability on the Amazon Reviews corpus over seven prefixes: 5M, 10M, 20M, 40M, 80M, 160M, and 227,899,533 rows, under a 58 GB limit and a 600-second per-query budget, with two warm-up runs and five measured runs per cell. The same five families are used; four keep the benchmark structure with rating labels, and the optional family uses `E D? C+ B*`. No query uses `PARTITION BY`, so every run scans a single partition in order, which removes the parallel work that partitioning would otherwise allow.

Table A.1 summarises each family averaged over the six sizes. Pattern structure, not match volume, drives the cost: `complex_nested` is the slowest family for all three systems, at 0.34 seconds for the engine against 2.52 for Oracle and 5.28 for Trino. The engine leads every family, with throughput from 2.58 million rows/sec on `complex_nested` to 8.13 million on `simple_sequence`.

Table A.2 groups the same cells by size. The engine records the lowest time in all 30 cells, and the margin holds as the input grows: 4.3–7.0× over Oracle and 9.3–13.3× over Trino.

Table A.3 gives the engine's time for every pattern–size cell. Execution time grows close to linearly with input size for all five families, and the per-family linear fits give $R^2 = 0.9904$–0.9997 over the measured range.

5.4.2. Throughput by Pattern and Size

Throughput in Table A.4 stays flat as the data grows, which is what near-linear execution time means in practice. It ranges from 2.49 to 9.40 million rows/sec and tracks pattern structure rather than dataset size.

5.4.3. Memory Metrics

Table A.5 shows the trade-off. Query memory rises with input size, from 6.9 MB at 100K rows to 163.1 MB at the full size, because the compiled path builds whole-column masks and intermediate arrays over all input rows rather than evaluating row by row. The deployment footprint reverses the order: the engine runs as a single Python process and peaks at 524 MB, against 2,057 MB for Oracle and 14,993 MB for Trino, whose table resides in the JVM heap.

5.4.4. Scalability on the large corpus

Table A.6 reports the second campaign. The engine and Oracle completed all 35 cells; Trino completed 33, because `complex_nested` exceeded the 600-second budget at 160M and 227,899,533 rows. The engine finished the full corpus on every family, from 24.8 seconds on `alternation` to 106.0 seconds on `complex_nested`, holding 2.2–9.2 million rows/sec across the range. Its resident footprint grows with the data and reaches 35.7 GB at the full corpus, so available memory is the practical limit for an in-memory engine.

5.4.5. Key Findings

Table 3 aggregates the first campaign. The engine averages 0.17 seconds and 6.11 million rows/sec per cell, against 1.04 seconds for Oracle and 1.98 for Trino, and it records the lowest time in all 30 cells while every cell agrees on the returned rows. The cost of that speed is per-query memory, where the engine is the highest of the three at 189.0 MB, while its total footprint stays the lowest. These results describe the evaluated single-node workload and the stated resource limits; they do not establish a general ranking of SQL engines.

6. Conclusions, Limitations and Future Work

Despite the system's comprehensive capabilities, several limitations remain. First, `complex pattern and quantifier interactions`: although the system supports concatenation, alternation, grouping, and standard quantifiers (`*`, `+`, `?`, `{n,m}`), certain combinations—particularly multiple greedy quantifiers nested within groups (e.g.,`(A+B*) +C?`)can trigger exponential state-space growth during automata construction. This issue primarily arises with three or more levels of nesting combined with unbounded quantifiers; by contrast, simpler patterns and bounded quantifiers behave efficiently. Second, `limited support for aggregate functions`: while a wide range of built-in aggregates (including conditional and statistical functions) is supported, the current implementation offers only limited support for user-defined aggregate functions.

We presented a SQL-in-`pandas` engine for executing `MATCH_RECOGNIZE` queries over DataFrames. That provides `SQL:2016 MATCH_RECOGNIZE` functionality for `pandas` DataFrames. bridging the gap between the expressiveness of relational queries and the flexibility of in-memory analytics, bringing SQL pattern matching capabilities to python data science workflows. This opens the door to unified and portable pipelines that preserve both semantics and developer productivity.
`MATCH_RECOGNIZE` allows data scientists and analysts to use powerful pattern-matching semantics directly within their familiar Pandas environment, without the need for complex Python code or external SQL engine dependencies. This reduces development complexity and enhances productivity for sequential data analysis across domains, including financial analysis, log processing, and time series pattern detection.

By addressing the identified limitations and implementing the future enhancements, our goal is to develop a more adaptable and efficient solution. That can handle complex pattern-matching scenarios across various data processing environments. Future work will focus on addressing current limitations through enhanced SQL clause support, distributed processing capabilities, and advanced analytics integration. This development roadmap outlined above provides a clear path for improving performance for the current implementation.

We plan to extend the engine through some updates, `Performance on Large Datasets:` The system performs efficiently on moderate-sized datasets but may require additional optimizations for large datasets; `Memory Usage for Large Patterns:` patterns with many variables and complex quantifiers can generate large automata that increase memory consumption; `Integration with Query Optimizers:`because the pattern-matching engine currently operates independently of database query optimizers, it may miss plan-level optimization opportunities.

Table 3: Overall Benchmark Statistics by System (Amazon UK Products)

System	Cells	Avg Time (sec)	Avg Throughput (rows/sec)	Max Query Mem (MB)	Max Footprint Mem (MB)
Proposed engine	30	0.17	6,110,392	189.0	524
Oracle 21c EE	30	1.04	1,156,106	58.7	2,057
Trino 473	30	1.98	602,098	20.8	14,993

Declarations

Ethical Approval. Not applicable

Funding. Not applicable

Data availability. The code used for this study is publicly available in the GitHub repository: https://github.com/MonierAshraf/Row_MATCH_RECOGNIZE. The dataset from the UK Amazon products marketplace used in the analysis is available from Kaggle: https://www.kaggle.com/datasets/asaniczka/amazon-uk-products-dataset-2023. In this study, nested prefixes of up to 2,222,742 rows from the Amazon UK Products dataset and up to 227,899,533 rows from the Amazon Reviews 2023 corpus (https://amazon-reviews-2023.github.io/) were used for experimental evaluation and performance testing. Additional methodological details and example usage are drawn from the Trino blog post `https://trino.io/blog/2021/05/19/row_pattern_matching.html` and from the article `https://link.springer.com/article/10.1007/s13222-022-00404-3`. All datasets and code referenced in this work are publicly available, and the links above provide direct access.

References

[1] D. Petković, Identifying possible financial frauds using sql row pattern recognition, International Journal of Computer Applications 184 (35) (2022) 31–34. doi:10.5120/ijca2022922446.
URL https://ijcaonline.org/archives/volume184/number35/32542-2022922446/

[2] F. Zemke, A. Witkowski, M. Cherniack, L. Colby, Pattern matching in sequences of rows, Technical report, ANSI Standard Proposal (2007).

[3] International Organization for Standardization, ISO/IEC TR 19075-5:2016 Information technology – Database languages – SQL Technical Reports – Part 5: Row Pattern Recognition in SQL , Feature R010: Row pattern recognition in FROM clause, `https://www.iso.org/standard/65143.html` (2016).

[4] International Organization for Standardization, International Electrotechnical Commission, Information technology — guidance for the use of database language sql — part 5: Row pattern recognition, `https://www.iso.org/standard/78936.html`, iSO/IEC 19075-5:2021, Edition 1 (2021).

[5] Oracle, Pattern recognition with match_recognize, `https://docs.oracle.com/en/middleware/fusion-middleware/osa/19.1/cqlreference/pattern-recognition-match_recognize.html` (2022).

[6] K. Hansen, Practical Oracle SQL: Mastering the Full Power of Oracle Database, Apress Berkeley, CA, 2020. doi:10.1007/978-1-4842-5617-6.

[7] Trino, Match_recognize, `https://trino.io/docs/current/sql/match-recognize.html` (2022).

[8] Trino, Row pattern recognition with match_recognize, `https://trino.io/blog/2021/05/19/row_pattern_matching.html` (2021).

[9] Snowflake Inc., Match_recognize clause, snowflake Documentation (2025).
URL `https://docs.snowflake.com/en/sql-reference/constructs/match_recognize`

[10] A. Flink, Match_recognize, `https://nightlies.apache.org/flink/flink-docs-master/docs/dev/table/sql/queries/match_recognize/` (2018).

[11] Google Cloud, MATCH_RECOGNIZE clause in GoogleSQL, bigQuery documentation (2025).
URL `https://cloud.google.com/bigquery/docs/reference/standard-sql/query-syntax#match_recognize_clause`

[12] Databricks, Databricks SQL release notes 2026, MATCH_RECOGNIZE beta in Databricks SQL and Runtime 19.0 (2026).
URL `https://docs.databricks.com/gcp/en/sql/release-notes/2026#match-recognize-is-now-in-beta`

[13] PostgreSQL Global Development Group, Unsupported features, postgreSQL 18 documentation (2025).
URL `https://www.postgresql.org/docs/18/unsupported-features-sql-standard.html`

[14] DuckDB Foundation, Development roadmap (2026).
URL `https://duckdb.org/roadmap`

[15] T. Findling, Bringing row pattern recognition to DuckDB: Translating MATCH_RECOGNIZE to WITH RECURSIVE (2025).

[16] Y. Cao, C.-Y. Chan, J. Li, K.-L. Tan, Optimization of analytic window functions (2012). arXiv:1208.0086.
URL `https://arxiv.org/abs/1208.0086`

[17] J. Michels, K. Hare, K. Kulkarni, C. Zuzarte, Z. H. Liu, B. Hammerschmidt, F. Zemke, The new and improved sql: 2016 standard, ACM Sigmod Record 47 (2) (2018) 51–60.

[18] D. Petković, Specification of row pattern recognition in the SQL standard and its implementations, Datenbank-Spektrum 22 (2) (2022) 163–174. doi:10.1007/s13222-022-00404-3.
URL https://doi.org/10.1007/s13222-022-00404-3

[19] G. Howe, Any interest in adding match_recognize?, message-ID: 7e495cb8-ca03-4e3e-aa7e-45bf75212da8@Spark (2020).
URL https://www.postgresql.org/message-id/7e495cb8-ca03-4e3e-aa7e-45bf75212da8@Spark

[20] MariaDB Foundation, Regular expressions overview, mariaDB Server Documentation (2025).
URL https://mariadb.com/docs/server/reference/sql-functions/string-functions/regular-expressions-functions/regular-expressions-overview

[21] E. Zhu, S. Huang, S. Chaudhuri, High-performance row pattern recognition using joins (2023). doi:10.14778/3579075.3579090.
URL https://doi.org/10.14778/3579075.3579090

[22] University of Tübingen, Database Systems Group, Match_recognize research project, database Systems Teaching Notes.
URL https://db.cs.uni-tuebingen.de/teaching/research-projects/match-recognize/

[23] University of Tübingen, Database Systems Group, Research projects, database Systems Teaching Notes.
URL https://db.cs.uni-tuebingen.de/teaching/research-projects/

[24] Š. Šprem, N. Tomažin, J. Matečić, M. Horvat, Building advanced web applications using data ingestion and data processing tools, Electronics 13 (4) (2024) 709.

[25] E. Georgiou, Evaluating Apache Flink, Kafka Streams, and Spark Streaming for scalable machine learning workflows (2025).

[26] M. Schule, H. Lang, M. Springer, A. Kemper, T. Neumann, S. Gunnemann, In-database machine learning with sql on gpus, in: Proceedings of the 33rd International Conference on Scientific and Statistical Database Management, SSDBM '21, Association for Computing Machinery, New York, NY, USA, 2021, p. 25–36. doi:10.1145/3468791.3468840.
URL https://doi.org/10.1145/3468791.3468840

[27] N. Makrynioti, R. Ley-Wild, V. Vassalos, sql4ml a declarative end-to-end workflow for machine learning, ArXiv abs/1907.12415 (2019).
URL https://api.semanticscholar.org/CorpusID:198968138

[28] D. Naphade, The evolution and modernization of data pipeline architectures, European Journal of Computer Science and Information Technology 13 (6) (2025) 42–53.

[29] J. Lucas, Y. Idris, B. Contreras-Rojas, J.-A. Quiane-Ruiz, S. Chawla, Rheemstudio: Cross-platform data analytics made easy, in: 2018 IEEE 34th International Conference on Data Engineering (ICDE), 2018, pp. 1573–1576. doi:10.1109/ICDE.2018.00179.

[30] A. Mozzillo, L. Zecchini, L. Gagliardelli, A. Aslam, S. Bergamaschi, G. Simonini, Evaluation of dataframe libraries for data preparation on a single machine (2025). doi:10.48786/EDBT.2025.27.
URL https://openproceedings.org/2025/conf/edbt/paper-96.pdf

[31] S. Chawla, B. Contreras Rojas, Z. Kaoudi, S. Kruse, J.-A. Quiané-Ruiz, Building your cross-platform application with RHEEM, arXiv preprint arXiv:1805.11723 (2018). doi:10.48550/arXiv.1805.11723.
URL https://arxiv.org/abs/1805.11723

[32] MonierAshraf, Row_match_recognize, gitHub repository (2025).
URL https://github.com/MonierAshraf/Row_match_recognize/blob/5d58e8de6b1143e8c1d77cd7017e37c7a8bae421/Examples%20Match_reocginze/Examples.ipynb

[33] J. Melton, A. Eisenberg, Sql:2016 support for row pattern recognition, SIGMOD Record 46 (2) (2017) 39–48. doi:10.1145/3105906.3105914.

[34] Y. S. Tarazona Bernal, Antlr 4 grammar of the swift 5 programing language (2021).

[35] T. Parr, The definitive ANTLR 4 reference, The Pragmatic Bookshelf, 2013.

[36] K. Salomaa, S. Yu, Nfa to dfa transformation for finite languages, in: International Workshop on Implementing Automata, Springer, 1996, pp. 149–158.

[37] ANTLR Team and Contributors, Trino sql grammar for antlr, https://github.com/antlr/grammars-v4/tree/master/sql/trino (2022).

[38] H. F. Hiroaki YAMAMOTO, A new finite automata construction using a prefix and a suffix of regular expressions, IEICE TRANSACTIONS on Information (2021). doi:10.1587/transinf.2020FCP0010.

[39] G. Xing, A simple way to construct nfa with fewer states and transitions, in: Proceedings of the 42nd Annual ACM Southeast Conference, ACMSE '04, Association for Computing Machinery, New York, NY, USA, 2004.

doi:10.1145/986537.986588.
URL https://doi.org/10.1145/986537.986588

[40] M. Ashraf, Python udf example for row pattern recognition, gitHub repository (2025).
URL https://github.com/MonierAshraf/Row_match_recognize/blob/master/Python_Examples/python_udf.ipynb

[41] asaniczka, Amazon UK Products Dataset 2023, Kaggle dataset, https://www.kaggle.com/datasets/asaniczka/amazon-uk-products-dataset-2023 (accessed 2025-07-16) (2023).

[42] Y. Hou, J. Li, Z. He, A. Yan, X. Chen, J. McAuley, Bridging Language and Items for Retrieval and Recommendation, Amazon Reviews 2023 dataset, McAuley Lab, UC San Diego, https://amazon-reviews-2023.github.io/ (accessed 2026-07-19) (2024).

Appendix A. Appendix

Appendix A.1. Detailed Benchmark Tables

The tables below give the per-pattern and per-size values behind the results reported in Section 5.4.

Appendix A.2. Full SQL Syntax Reference

```
SELECT <select list>
FROM <source table>
MATCH_RECOGNIZE
(
  [ PARTITION BY <partition list> ]
  [ ORDER BY <order by list> ]
  [ MEASURES <measure list> ]
  [ ONE ROW PER MATCH | ALL ROWS PER MATCH ]
  [ AFTER MATCH <skip-to-option> ]
  PATTERN ( <row pattern> )
  [ SUBSET <subset list> ]
  DEFINE <definition list>
) AS <table alias>;
```

Appendix A.3. Step-by-Step Execution Analysis

1. **Query Parsing: In this step SQL parser extracts components from the query.**
   Input: Raw SQL query string Process: Regular expression matching to extract each clause
   Output: Dictionary with parsed components.
   The parser extracts:

   - `PARTITION BY: customer_id`
     To group the data by customer_id, meaning pattern matching will be performed separately for each customer
   - `ORDER BY: order_date`
     To sort data within each partition chronologically by order_date to establish sequence order
   - `MEASURES:`
     - `start_date = A.order_date`
     - `end_date = C.order_date`
     - `bottom_price = C.price`

     To define what values to extract from matched patterns. Each measure specifies which variable's column value to return in the result.
   - `PATTERN: (A B C)`
     To specify the sequence pattern to match exactly 3 consecutive rows labeled A, B, and C
   - `DEFINE:`
     - `B AS B.price < A.price`
     - `C AS C.price < B.price`

   To define conditions for each pattern variable must be satisfied.
   B must have lower price than A and C must have a lower price than B.

2. **Data Partitioning**
   To groupby operation splits the DataFrame into separate groups based on the PARTITION BY columns.
   Input: partition_by columns: df.groupby(['customer_id'])
   Output: Two separate DataFrames, one per customer.

Group 1 – cust_1

customer_id	order_date	price
cust_1	2020-05-11	100
cust_1	2020-05-12	200
cust_1	2020-05-14	100
cust_1	2020-05-16	50
cust_1	2020-05-17	100

Group 2 – cust_2

customer_id	order_date	price
cust_2	2020-05-13	8
cust_2	2020-05-15	4
cust_2	2020-05-18	6

3. **Pattern Matching for `cust_1`**
   Tries to match the pattern starting from each possible row position.

   - *Attempt 1 (row 0, price=100):*
     Try to build pattern A-B-C starting from the first row
     - A = row 0 (100):Row 0 (date=2020-05-11, price=100)
     - B: row 3 (50): Row 3 (date=2020-05-16, price=50)
       Look for row where $B.price < A.price$ (price < 100)
       Index 1: price=200 → 200 < 100? FALSE
       Index 2: price=100 → 100 < 100? FALSE
       Index 3: price=50 → 50 < 100? TRUE

Table A.1: Pattern Performance Summary (5 Patterns × 6 Dataset Sizes = 30 Tests per System)

Pattern	Engine Throughput (rows/sec)	Engine (sec)	Oracle (sec)	Trino (sec)
simple_sequence	8,134,451	0.10	0.73	1.27
alternation	5,445,348	0.15	0.53	1.04
quantified	7,751,889	0.11	0.85	1.35
optional_pattern	6,637,948	0.13	0.59	0.95
complex_nested	2,582,321	0.34	2.52	5.28

Table A.2: Performance Summary by Dataset Size

Dataset Size (rows)	Engine (sec)	Oracle (sec)	Trino (sec)	Oracle/ Engine	Trino/ Engine
100,000	0.022	0.098	0.257	4.3×	11.4×
200,000	0.040	0.178	0.382	4.5×	9.6×
400,000	0.078	0.381	0.725	4.9×	9.3×
800,000	0.150	0.987	1.892	6.6×	12.6×
1,600,000	0.287	2.001	3.822	7.0×	13.3×
2,222,742	0.426	2.622	4.788	6.2×	11.2×

Table A.3: Engine Execution Time (seconds) by Pattern and Dataset Size

Pattern	Dataset Size (rows)					
	100K	200K	400K	800K	1.6M	2.22M
simple_sequence	0.015	0.025	0.049	0.093	0.170	0.269
alternation	0.023	0.040	0.072	0.141	0.265	0.373
quantified	0.016	0.028	0.053	0.089	0.178	0.281
optional_pattern	0.018	0.031	0.063	0.110	0.218	0.331
complex_nested	0.040	0.076	0.152	0.317	0.604	0.875

Table A.4: Engine Throughput (million rows/sec) by Pattern and Dataset Size

Pattern	Dataset Size (rows)					
	100K	200K	400K	800K	1.6M	2.22M
simple_sequence	6.55	8.01	8.05	8.56	9.40	8.23
alternation	4.40	4.98	5.57	5.71	6.05	5.96
quantified	6.12	7.05	7.46	8.96	8.99	7.94
optional_pattern	5.58	6.54	6.35	7.29	7.34	6.72
complex_nested	2.49	2.64	2.63	2.53	2.65	2.54

  - C: Look for row where $C.price < B.price$ ($price < 50$)
    Index 4: price=100 → 100 < 50? FALSE
    no price < 50 after row 5, ⇒ *no match*
    No valid C found, PATTERN MATCH FAILED

- *Attempt 2 (row 1, price=200):*
  - A = row1 (200): Row 1 (date=2020-05-12, price=200)
  - B = row3 (100) :Row 2 (date=2020-05-14, price=100)
    Index 2: price=100 → 100 < 200? TRUE
  - C =row5 (50): Row 3 (date=2020-05-16, price=50)
    Index 3: price=50 → 50 < 100? TRUE
    row5 (50) ⇒ *match found (rows 1, 3, 5)*
    Match Dictionary: {'A': 1, 'B': 2, 'C': 3}

- *Attempt 3 (row 3, price=100):*
  - A = row3 (100) :Row 2 (date=2020-05-14, price=100)
  - B = row5 (50) : Row 3 (date=2020-05-16, price=50)
    Index 3: price=50 → 50 < 100? TRUE
  - C: no *price* < 50 after row5 ⇒ *no match*
    Index 4: price=100 → 100 < 50? FALSE
    PATTERN MATCH FAILED

Table A.5: Memory by Dataset Size: Engine Query Peak and Deployment Footprints (MB)

Dataset Size (rows)	Engine Query Peak	Engine Footprint	Oracle Footprint	Trino Footprint
100,000	6.9	87	1,982	8,676
200,000	15.1	106	1,965	9,167
400,000	29.4	142	1,978	9,791
800,000	59.0	218	1,986	11,024
1,600,000	113.7	351	2,024	13,253
2,222,742	163.1	486	2,050	14,957

Table A.6: Engine Execution Time (seconds) on the Amazon Reviews Corpus

Pattern	Dataset Size (rows)						
	5M	10M	20M	40M	80M	160M	227.9M
`simple_sequence`	0.6	1.3	2.7	5.3	10.0	19.5	27.8
`alternation`	0.7	1.4	2.7	5.3	9.4	19.0	24.8
`quantified`	0.6	1.4	2.7	5.4	10.1	20.9	29.3
`optional_pattern`	0.7	1.6	3.1	6.2	11.6	23.9	33.8
`complex_nested`	2.3	4.8	9.5	19.5	38.8	77.0	106.0

4. **Pattern Matching for** `cust_2`

   - *Attempt 1 (row 3, price=100):*
     - A = row0 (8) : Row 0 (date=2020-05-13, price=8)
     - B = row1 (4) : Row 1 (date=2020-05-15, price=4)
       (Look for row where *B.price* < *A.price* (*price* < 8)
       Index 1: price=4 → 4 < 8? TRUE
     - C: Look for row where *C.price* < *B.price* (*price* < 4)
       Index 2: price=6 → 6 < 4? FALSE
       No valid C found, PATTERN MATCH FAILED

   - *Attempt 2 (row 4, price=4):* no B ⇒ no match
     - A = row1 (4): Row 1 (date=2020-05-15, price=4)
     - B: Look for row where *B.price* < *A.price* (*price* < 4)
       Index 2: price=6 → 6 < 4? FALSE
       No valid B found, PATTERN MATCH FAILED
     - C: cannot form a complete 3-row pattern
       PATTERN MATCH FAILED

5. **Apply Measures**
   The evaluate_measure function extracts specific column values from the matched pattern rows according to the MEASURES clause.
   Only one match: `cust_1` at rows 1, 3, 5.
   input:Match dictionary: 'A': 1, 'B': 2, 'C': 3 (from cust_1)

   Measures: `{'start_date': 'A.order_date', 'end_date': 'C.order_date', 'bottom_price': 'C.price'}`

   start_date = A.order_date Extract `order_date` from the row matched to variable A.
   A is at index 1: `df.iloc[1]['order_date']` = `'2020-05-12'`.

   end_date = C.order_date Extract `order_date` from the row matched to variable C.
   C is at index 3: `df.iloc[3]['order_date']` = `'2020-05-16'`.

   bottom_price = C.price Extract `price` from the row matched to variable C.
   C is at index 3: `df.iloc[3]['price']` = 50.

   customer_id The partition key identifies which group this match belongs to Value: `'cust_1'`.

6. **Outputs:**

   Our goal is to represent a detected pattern in customer behavior.

   Pattern Meaning:.

   - **Customer:** `cust_1`
   - **Pattern Type:** Consecutive price decline ($A > B > C$)
   - **Time Period:** 2020-05-12 to 2020-05-16 (4 days)
   - **Price Sequence:** 200 → 100 → 50
   - **Business Insight:** Customer `cust_1` exhibited a consistent downward trend in order values.

   The final step assembles all successful matches into a result `DataFrame`.

   Input: List of result rows (one per successful match)

Process: `pd.DataFrame(results)`

Output: Final result `DataFrame`

customer_id	start_date	end_date	bottom_price
`cust_1`	2020-05-12	2020-05-16	50

This reflects a consistent downward trend:

- 200 on 2020-05-12 → 100 on 2020-05-14 → 50 on 2020-05-16